

Grado en Ingeniería Informática
Sistemas Interactivos y Ubicuos
Curso 2025 / 2026

Entrega 2 – Prototipar



Grupo 82 – Equipo 3

Carmen Peláez Martín

100522094@alumnos.uc3m.es

Juan Francisco Salor Búrdalo

100522369@alumnos.uc3m.es

Ana Sanz del Collado

100522234@alumnos.uc3m.es

ÍNDICE

1. INTRODUCCIÓN	3
2. ESPECIFICACIONES TÉCNICAS	3
Arquitectura del sistema (diagramas de componentes y comunicación):	3
Tecnologías utilizadas	5
3. IMPLEMENTACIÓN DE LOS PROTOTIPOS	6
Descripción y justificación de las interacciones implementadas	6
Videos que muestran el prototipo en funcionamiento	8
4. PROTOTIPADO EXPERIENCIAL	9
Descripción de las sesiones realizadas	9
Reflexión sobre cómo influyeron en la implementación del prototipo	10
Videos de las sesiones de prototipado y análisis de los hallazgos obtenidos	10
5. INTERACCIONES Y MEJORAS	10
Problemas encontrados y cómo se abordaron	10
Cambios realizados respecto a las ideas originales	11
Nuevas ideas generadas	12
6. REFLEXIÓN FINAL	13
Aprendizajes obtenidos y próximos pasos hacia la evaluación en P3	13
7. USO DE IA Y BIBLIOGRAFÍA	13

1. INTRODUCCIÓN

El proyecto consiste en un agente interactivo de cocina diseñado para guiar a los usuarios en la preparación de recetas de forma ubicua. El objetivo principal es ofrecer una experiencia de manos libres en un entorno donde los usuarios suelen tener las manos manchadas u ocupadas. Para lograr esto, el sistema reemplaza las interacciones tradicionales de la pantalla táctil por un sistema basado en el reconocimiento de voz, la detección de gestos y la presencia corporal.

El sistema es ubicuo y distribuido, ya que permite separar el dispositivo que actúa como sensor (teléfono móvil) del dispositivo que actúa como pantalla interactiva (ordenador), sincronizándose en tiempo real

2. ESPECIFICACIONES TÉCNICAS

Arquitectura del sistema (diagramas de componentes y comunicación):

Diagrama de Componentes

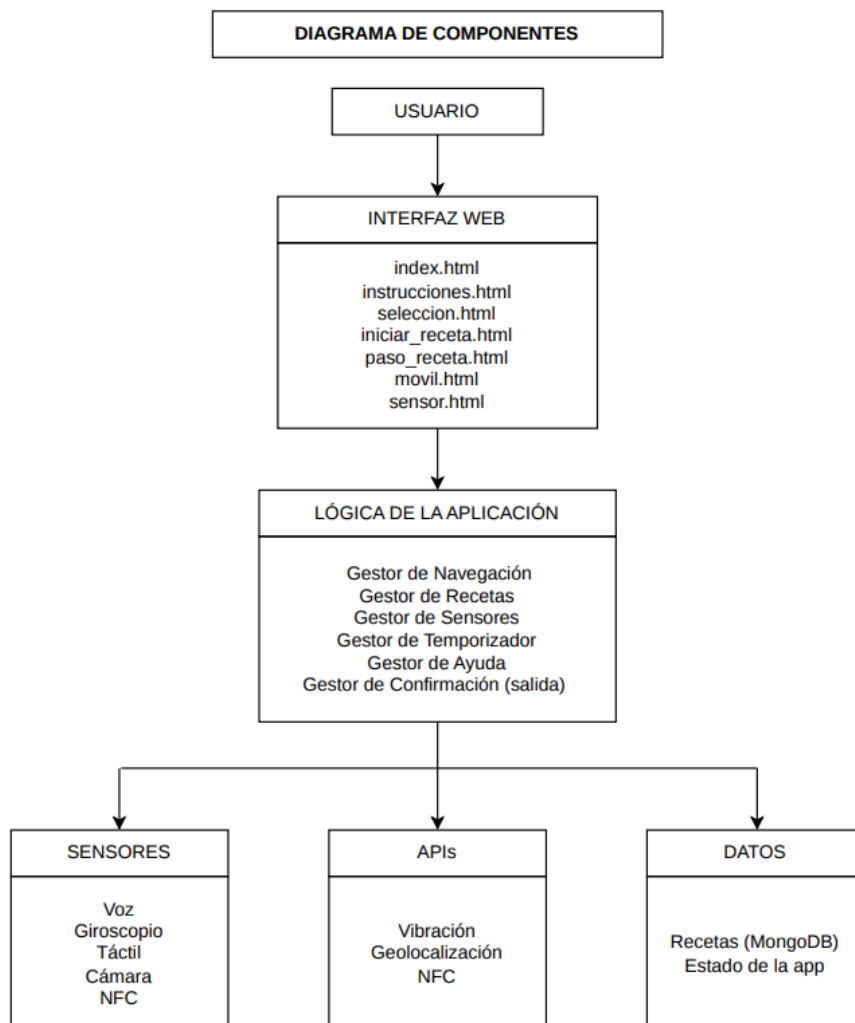
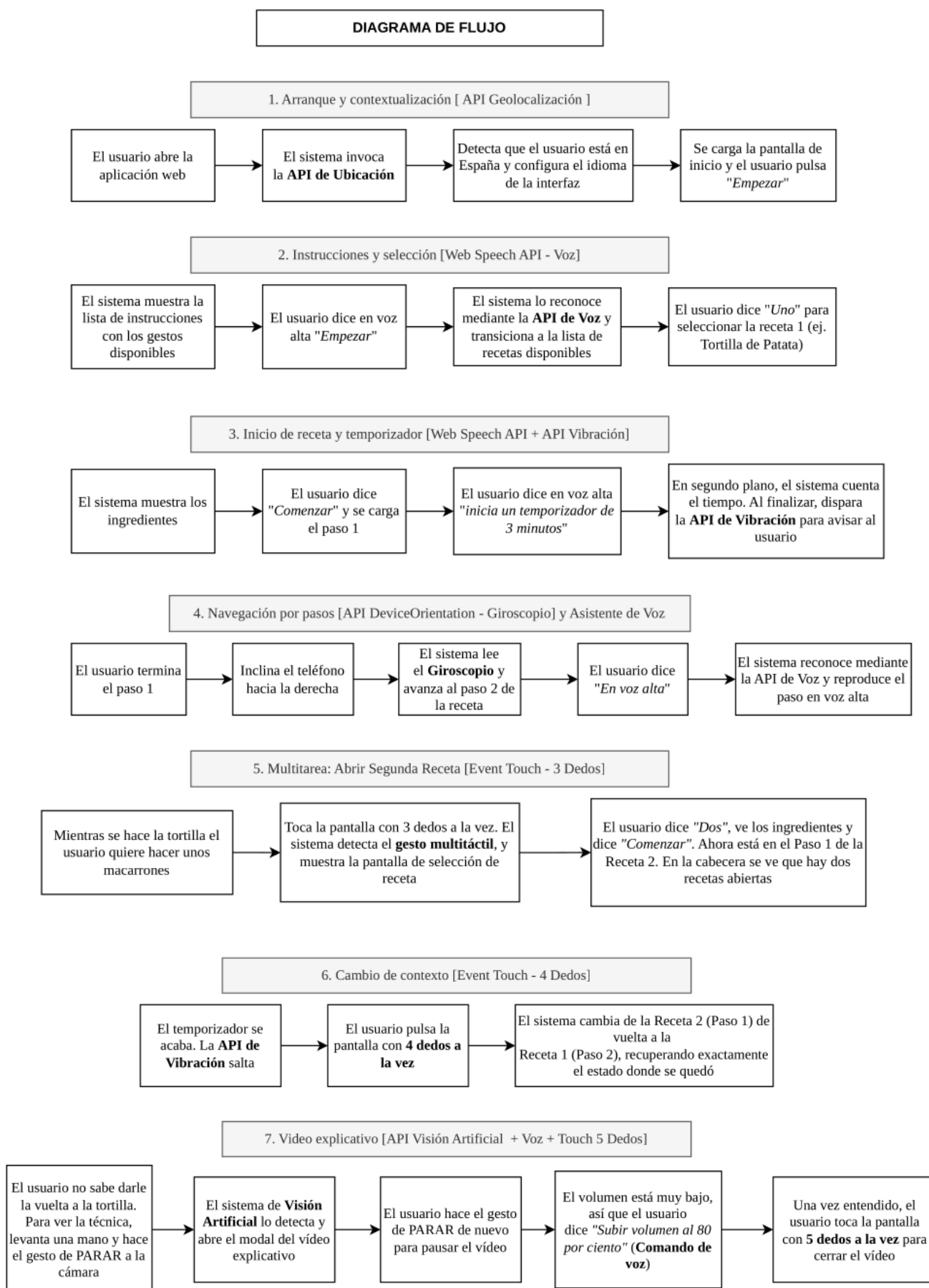
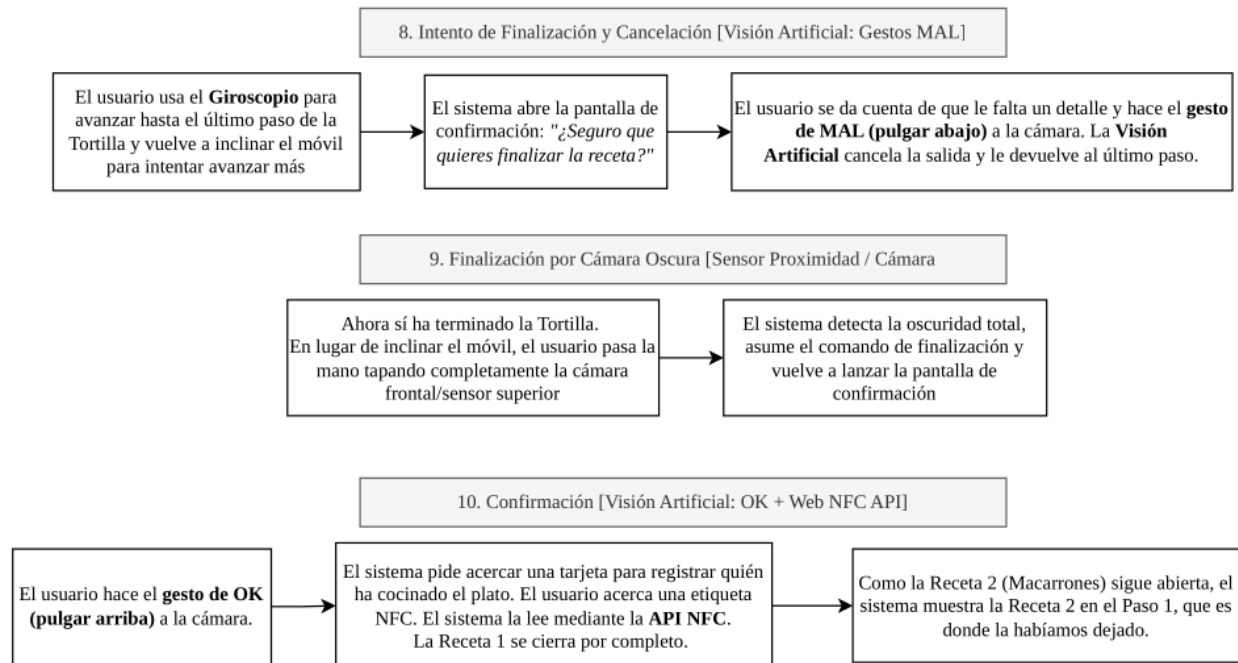


Diagrama de flujo y comunicación (ejemplo)





Tecnologías utilizadas

Para implementar este sistema ubicuo en tiempo real, hemos incorporado tecnologías que permiten integrar visión artificial y reconocimiento de voz directamente en el navegador de forma distribuida

- Node.js y Express: conforman el núcleo del servidor backend, encargándose de gestionar las peticiones HTTP estáticas y proveer el entorno de ejecución necesario para establecer las conexiones de red locales
- Socket.IO: permite la comunicación bidireccional en tiempo real entre el dispositivo que captura los gestos y los dispositivos que muestran la interfaz
- MongoDB: Base de datos utilizada para almacenar la información estructurada de las recetas (título, pasos, ingredientes, enlaces a vídeos, etc.)
- Handtrack.js: Librería que permite detectar la posición y movimiento de las manos en el navegador
- Annyang (Web Speech API): librería que facilita el reconocimiento de voz continuo en el navegador, permitiendo mapear frase y palabras a funciones de control en la aplicación
- SpeechSynthesis (Web API): API nativa del navegador para convertir texto a voz (Text-to-Speech), permitiendo que el sistema lea los pasos de la receta en voz alta
- YouTube IFrame Player API: Permite controlar (play, pausa, volumen, salto de tiempo) los vídeos de ayuda de cada paso en la receta

3. IMPLEMENTACIÓN DE LOS PROTOTIPOS

Descripción y justificación de las interacciones implementadas

Dado que el entorno culinario exige que el usuario tenga las manos ocupadas o manchadas, hemos sustituido las interfaces táctiles tradicionales por un sistema de interacción multimodal basado en el reconocimiento de voz, la detección de gestos y el uso de sensores integrados en el dispositivo móvil. A continuación, se detallan las funcionalidades implementadas y su justificación técnica:

1. Navegación y selección de recetas (Web Speech API - Annyang)

La interacción se realiza íntegramente mediante la voz. El usuario puede navegar por el menú y seleccionar recetas pronunciando su número identificador (por ejemplo, "Uno" o "Dos"). Esta decisión de diseño elimina la barrera física de tocar la pantalla al inicio del proceso, garantizando una experiencia manos libres desde el primer contacto.

El usuario también puede pedir ayuda en cualquier momento diciendo “Ayuda”. El sistema mostrará la pantalla de instrucciones donde se ven todos los gestos disponibles y al regresar, vuelve exactamente a la pantalla desde la que se solicitó la ayuda, preservando el estado de la receta

2. Control de funciones principales (Device Orientation API - Giroscopio)

Para la navegación dentro de una receta se ha integrado la API Device Orientation. El móvil actúa como sensor remoto, al inclinarlo hacia la derecha, Socket.IO emite un evento al servidor que avanza al paso siguiente, al inclinarlo hacia la izquierda retrocede al paso anterior. Esto ofrece una interacción física sin necesidad de contacto visual directo con la pantalla.

3. Salida o confirmación (Sensores Ambientales + Gestos + Web NFC API // Giroscopio + Gestos + Web NFC API)

El proceso de salida / cierre de la receta se puede activar de dos formas: llegando al último paso de la receta e intentando avanzar, o tapando la cámara en cualquier paso. En ambos casos, el sistema muestra una pantalla de confirmación con la pregunta “¿Seguro que quieres finalizar la receta?”. En la pantalla de confirmación, Media Pipe detecta el gesto de la mano:

- Pulgar arriba: confirma la salida y solicita una tarjeta NFC para registrar la sesión
- Pulgar abajo: cancela la salida y vuelve al paso en el que se encontraba el usuario

Si el dispositivo no dispone de lector NFC, la confirmación se puede completar sin este paso. Para la detección de cámara oscura, el sistema calcula el brillo medio de los píxeles del frame, si este valor desciende por debajo de un umbral definido (UMBRAL_OSCURIDAD = 15) de forma continua, se dispara el evento de confirmación de salida

4. Implementación de funcionalidades adicionales

a. Multitarea - Gestión de recetas paralelas (Touch API)

El sistema permite abrir y gestionar múltiples recetas simultáneamente almacenando el estado de cada una en la memoria local. Mediante gestos multitáctiles específicos, el sistema permite cocinar varios platos a la vez y alternar entre ellos, guardando el progreso exacto de cada uno. Los dos gestos táctiles implementados son:

- Toque con 3 dedos simultáneos: invoca el menú de selección para añadir una nueva receta sin cerrar la receta activa
- Toque con 4 dedos simultáneos: alterna entre recetas activas, recuperando el paso exacto de cada una

La cabecera de la interfaz muestra en todo momento cuántas recetas están activas. Al cerrar una receta, el sistema recupera automáticamente la siguiente receta abierta en el paso donde se había quedado

b. Asistente de voz y temporizador por voz (Speech Synthesis API)

Integrando la API SpeechSynthesis Utterance, el sistema es capaz de leer las instrucciones de la receta en voz alta. El usuario activa o desactiva esta función mediante los comandos de voz "En voz alta" y "Silencio"

Adicionalmente, se implementó un procesador de lenguaje natural que extrae variables numéricas de frases como "Inicia un temporizador de X segundos/minutos", lanzando una cuenta atrás en segundo plano que finaliza activando la API de Vibración del hardware para alertar al usuario

c. Vídeo explicativo con control multimodal (Youtube IFrame Player API + Gestos + Voz)

Desde cualquier paso de la receta, el usuario puede abrir un video explicativo levantando la palma abierta frente a la cámara (gesto STOP detectado por Media Pipe). Dentro del reproductor, el usuario dispone de las siguientes interacciones:

- Gesto de STOP con una mano: pausa o reanuda el vídeo
- Comando de voz "volumen al X por ciento": ajusta el volumen del vídeo al porcentaje indicado mediante la Youtube IFrame Player API
- Toque con 5 dedos simultáneos: cierra el video y vuelve al paso de la receta

d. Geolocalización e idioma automático (Geolocation API)

Al arrancar la aplicación, la Geolocation API detecta el país en el que se encuentra el usuario. El sistema configura automáticamente el idioma de toda la interfaz, garantizando que el usuario de cada país vea la aplicación en el idioma correspondiente. Esta funcionalidad se ejecuta de forma transparente en segundo plano, sin requerir ninguna acción por parte del usuario

e. Vibración del Temporizador Inteligente (Vibration API)

Cuando el temporizador por voz llega a cero, el sistema activa la Vibration API del hardware del móvil. El patrón de vibración diseñado alerta al usuario de forma inequívoca (varias vibraciones cortas interrumpidas), permitiéndole saber que el tiempo ha concurrido sin necesidad de mirar a la pantalla

f. Identificación de sesión con NFC (Web NFC PI)

Al confirmar el fin de una receta, el sistema solicita al usuario que acerque una tarjeta al lector NFC del móvil. La Web NFC API lee el identificador de la tarjeta y lo asocia a la sesión finalizada, permitiendo llevar un registro de qué usuario ha cocinado cada receta. En dispositivos móviles sin NFC, este paso se omite y la receta se cierra igualmente

Videos que muestran el prototipo en funcionamiento

[Video del funcionamiento](#)

4. PROTOTIPADO EXPERIENCIAL

Descripción de las sesiones realizadas

Para la elaboración del trabajo podemos diferenciar dos tipos de sesiones de trabajo:

Sesiones en clase: asistir a clase y realizar las entregas semanales nos ha permitido entender e ir al día con las tecnologías que luego usaremos en el proyecto. Los ejercicios semanales nos han servido para reutilizar código de APIs o ideas a la hora de hacer el proyecto final

Sesiones privadas del equipo: reuniones periódicas para organizar el trabajo de la semana y asignar las tareas a cada miembro del equipo. Estas sesiones privadas del equipo las podemos dividir en:

- Fase 1: Diseño de la interfaz y lógica de navegación: una vez teníamos la idea elegida de la anterior fase del proyecto (P1) tocaba diseñar la interfaz de usuario. Utilizamos la herramienta Figma para diseñar todas las pantallas. Nos ha resultado una herramienta muy útil ya que nos ha permitido diseñar la página de una forma uniforme.
- Fase 2: Decisión de la arquitectura del proyecto: Una vez ya teníamos la idea clara decidimos de forma unánime como íbamos a organizar las carpetas del proyecto y el estilo de programar para que fuera lo más uniforme posible. Quisimos hacer mucho hincapié en programar basándose en el Principio de Responsabilidad Única, algo básico en programación.
- Fase 3: Desarrollo de la interfaz visual: con el diseño de Figma realizado en la fase 1 como referencia, empezamos a desarrollar el html de todas las pantallas de nuestra aplicación
- Fase 4: Integración de APIs básicas: después de diseñar toda la parte visual, comenzamos a programar las funcionalidades básicas e implementar cosas como Socket-io (conectar móvil con PC) o Web Speech API (reconocimiento de voz)
- Fase 5: Integración de APIs Avanzadas: para cumplir con los requisitos de interacción multimodal, añadimos APIs complementarias: acceso al giroscopio del móvil (Device Orientation) para cambiar de paso inclinando el dispositivo o NFC para confirmar el fin de la receta
- Fase 6: Pruebas de Usabilidad y Depuración: la fase final consistió en someter el sistema a pruebas reales. Esto nos permitió afinar la sensibilidad de los sensores, corregir bloqueos en la navegación y pulir el manejo de eventos simultáneos

Reflexión sobre cómo influyeron en la implementación del prototipo

La estructuración del trabajo en estas fases fue clave para que el prototipo evolucionara sin que el código colapsara. Las sesiones iniciales de diseño y arquitectura (Fases 1, 2 y 3) nos permitieron aplicar el Principio de Responsabilidad Única, separando estrictamente la interfaz visual de la lógica de cada sensor. Disponer de un guión visual (modelo de figma) antes de escribir código aceleró significativamente el desarrollo posterior

Esta base modular fue lo que nos permitió afrontar las Fases 4 y 5 con seguridad, integrando primero la comunicación básica (Socket.io y voz) y sumando después sensores avanzados (gestos, giroscopio, NFC, táctil, etc.) sin que unos interfirieran con otros

Finalmente, la fase de pruebas (Fase 6) fue la que conectó la teoría con la realidad: testear el prototipo nos hizo darnos cuenta de que necesitábamos añadir tiempos de bloqueo (cooldowns) para evitar que los gestos se repitieran sin control, y nos obligó a implementar escudos en el código para que el sistema operativo no bloqueara la pantalla táctil, logrando así un producto verdaderamente estable y usable

Videos de las sesiones de prototipado y análisis de los hallazgos obtenidos

[Video de los hallazgos obtenido](#)

5. INTERACCIONES Y MEJORAS

Problemas encontrados y cómo se abordaron

Durante la integración de las distintas tecnologías, nos enfrentamos a varios desafíos técnicos que requirieron replantear algunas soluciones:

Conflictos de red y túneles inestables: Inicialmente usamos herramientas como Local Tunnel para exponer nuestro servidor local a internet y conectar el móvil. Sin embargo, los servidores gratuitos generaban cuelgues, bloqueaban la base de datos (MongoDB) y provocaban que la página no cargara. Lo solucionamos descartando el túnel y conectando los dispositivos a través de la red local (IP), forzando los permisos de seguridad (HTTPS) desde la configuración interna del navegador (chrome://flags)

Falsos positivos en la IA de gestos: Al implementar la detección de manos con Media Pipe, notamos que el sistema confundía gestos. Por ejemplo, al intentar cerrar el puño, la IA leía la pequeña elevación del dedo y activaba erróneamente la orden de "Pulgar arriba" (Confirmar). Lo solucionamos reescribiendo la lógica matemática de la detección, priorizando la lectura global de todos los dedos flexionados antes de evaluar pulgares individuales, logrando que el gesto del puño cerrara correctamente los vídeos

Conflicto de la Touch API con el Sistema Operativo: Al implementar gestos táctiles de 3 y 4 dedos para cambiar de receta, descubrimos que el sistema operativo Windows "secuestraban" el gesto para minimizar ventanas o cambiar de aplicación antes de que la página web pudiera leerlo. Abordamos esto cambiando el evento de touchstart a touchend y usando preventDefault(). Finalmente, para la presentación del proyecto, optamos por desactivar temporalmente los atajos táctiles del sistema operativo, garantizando una interacción web fluida y sin interrupciones

Limitaciones de seguridad del navegador (Bluetooth): Teníamos la intención de usar la Web Bluetooth API para que la interfaz cambiara automáticamente al conectar un dispositivo externo. Tras investigar la documentación, descubrimos que los navegadores bloquean la escucha pasiva por motivos estrictos de privacidad, requiriendo siempre un emparejamiento manual mediante un botón y una ventana emergente. Al no encajar en la experiencia de usuario "sin manos" que buscábamos, descartamos el Bluetooth y lo sustituimos por la integración de la Web NFC API y la Touch API, lo que nos permitió cumplir holgadamente con los requisitos técnicos del proyecto manteniendo la usabilidad

Interferencias ambientales en el reconocimiento de voz (El test en el aula): Durante la fase de pruebas, decidimos testear el prototipo en el entorno de la clase. Al intentarlo, el sistema empezó a comportarse de forma errática, ignorando comandos o activando funciones sin que lo pidiéramos. En un primer momento nos preocupamos pensando que el servidor o la lógica de la aplicación se habían roto. Sin embargo, al analizar los registros (logs), descubrimos que el micrófono estaba captando el ruido de fondo y las conversaciones de otros compañeros, lo que confundía a la API.

Cambios realizados respecto a las ideas originales

En la entrega anterior, el equipo votó y seleccionó un conjunto de interacciones como punto de partida para el prototipo. Sin embargo, durante la implementación y las pruebas físicas, varias de esas ideas originales tuvieron que ser modificadas o descartadas al encontrar limitaciones técnicas. A continuación se describe cada cambio y su justificación:

- **Navegación por pasos (Giroscopio vs Visión):** Se sustituyó el gesto manual (izq-dcha) por el giroscopio del móvil. La lectura directa del giroscopio hardware eliminó la ambigüedad y los falsos positivos de la cámara
- **Gestión multitarea (Táctil vs Gesto 2 manos):** Se descartó el uso de dos manos frente a la cámara para alternar recetas por romper el flujo de cocina. Se implementaron gestos de 4 dedos sobre la pantalla
- **Audio y Visibilidad:** Se eliminaron los automatismos de zoom y volumen por ruido ambiental al generar imprecisiones

- Conectividad (NFC vs Bluetooth): Se descartó Bluetooth por las restricciones de privacidad del navegador que exigen clics manuales. Se sustituyó por Web NFC API para una interacción pasiva coherente con el entorno "manos libres"
- Selección (Voz vs táctil): Se eliminó la necesidad de tocar la pantalla para iniciar o elegir recetas. Ahora se utilizan comandos de voz como "Empezar" y "Comenzar" para mantener la higiene
- Feedback del Temporizador: El sistema ahora activa la API de Vibración al finalizar la cuenta atrás. Esto permite alertar al usuario mediante una respuesta física sin necesidad de que se encuentre mirando a la pantalla

Nuevas ideas generadas

- Confirmación de seguridad (OK / MAL): Para evitar cierres accidentales al tapar la cámara, se añadió una pantalla de confirmación controlada por gestos de pulgar arriba (confirmar) o pulgar abajo (cancelar)
- Control de vídeo (Pausa/Reanuda): Se reutilizó el gesto de "PARAR" (mano abierta) frente a la cámara para pausar y reanudar el vídeo explicativo sin tocar el dispositivo
- Volumen por voz: Se implementó el comando "Subir volumen al X por ciento". Esto permite ajustar el audio del vídeo mediante lenguaje natural para adaptarse al ruido de la cocina
- Apertura de nueva receta: Se añadió la funcionalidad de invocar el menú de selección mediante un gesto de 3 dedos. Esto permite añadir platos al flujo de trabajo en cualquier momento
- Cierre de vídeo con 5 dedos: Se añadió este gesto táctil diferencial para cerrar el modal de ayuda visual de forma rápida e inequívoca
- Ayuda Global por Voz: Se creó un estado de interrupción mediante el comando "Ayuda", permitiendo consultar los gestos en cualquier momento y regresar al punto exacto de la receta sin perder el progreso
- Contextualización por Ubicación: Se utiliza la API de Geolocalización al arrancar para detectar el país del usuario. El sistema configura automáticamente el idioma de la interfaz según su posición

6. REFLEXIÓN FINAL

Aprendizajes obtenidos y próximos pasos hacia la evaluación en P3

El desarrollo de esta fase P2 nos ha demostrado el potencial de las Web APIs modernas para crear sistemas de computación ubicua sin depender de un hardware complejo. Hemos comprendido que la clave de una interfaz invisible no reside en la complejidad de los gestos, sino en la naturalidad y robustez de la respuesta del sistema

La distribución del sistema entre móvil y ordenador resultó ser una de las decisiones más acertadas del proyecto. Permite al usuario mantener el móvil en la encimera mientras interactúa con la pantalla del ordenador, creando una separación natural entre el espacio de cocina y el espacio de información

Para la fase P3 (Validación), nuestros próximos pasos se centrarán en:

- evaluación con usuarios reales: diseñaremos y llevaremos a cabo sesiones de test de usabilidad con participantes externos al equipo, utilizando las metodologías vistas en clase
- análisis de métricas de interacción: mediremos tiempos de respuesta, tasas de éxito de cada gesto y el número de intentos necesarios para completar cada acción
- interacción basada en feedback: incorporaremos las mejoras derivadas de las sesiones de evaluación, priorizando la robustez del reconocimiento de voz en entornos ruidosos y la claridad de los indicadores visuales en cada interacción

7. USO DE IA Y BIBLIOGRAFÍA

Uso de IA

Se han utilizado herramientas de Inteligencia Artificial (ChatGPT, Claude, Gemini) como apoyo al desarrollo, específicamente para:

- Comprobar la robustez del código y buscar algún fallo para solucionarlo evitando que nos pille desprevenido el día de la presentación.
- Documentación sobre el funcionamiento de las APIs y como podíamos implementarlas en el código
- Revisar y mejorar la redacción de la memoria, asegurando coherencia y claridad en la exposición del proceso

En todos los casos, la IA se limitó al apoyo y consulta. Las decisiones de diseño, arquitectura e implementación fueron tomadas íntegramente por el equipo

Bibliografía

- Documentación oficial de MongoDB: <https://www.mongodb.com/es/docs/>
- Documentación oficial de Socket.IO: <https://socket.io/docs/v4/>
- Documentación oficial de MediaPipe Hands:
- Documentación oficial de Handtrack.js: <https://github.com/victordibia/handtrack.js/>
- Documentación de Annyang: <https://github.com/TalAter/annyang>
- Web NFC API: https://developer.mozilla.org/en-US/docs/Web/API/Web_NFC_API
- Vibration API: https://developer.mozilla.org/es/docs/Web/API/Vibration_API
- Geolocation API: https://developer.mozilla.org/es/docs/Web/API/Geolocation_API
- YouTube IFrame Player API:
https://developers.google.com/youtube/iframe_api_reference
- SpeechSynthesis API:
<https://developer.mozilla.org/en-US/docs/Web/API/SpeechSynthesis>
- Device Orientation API:
<https://developer.mozilla.org/en-US/docs/Web/API/DeviceOrientationEvent>